User Defined Functions and Structures

4	User defined function and Structures		14	CO4
	4.1	User Defined Functions: Need, Function Declaration and Definition, Return Values, Function Calls, Passing Arguments to a Function by Value, Recursive functions		
	4.2	Structures and Unions: Introduction, Declaring and defining Structure, Structure Initialization, Accessing and Displaying Structure Members, Array of Structures, Introduction to Unions, Structure Vs Unions		

User defined Function

- A function is a self-contained block of statements that perform a coherent task of some kind. Every C program can be thought of as a collection of these functions. As we noted earlier, using a function is something like hiring a person to do a specific job for you.

```
# include <stdio.h>
void message( ) ; /* function prototype declaration */
int main( )
{
message( ) ; /* function call */
printf ( "Cry, and you stop the monotony!\n" ) ;
return 0 ;
}
void message( ) /* function definition */
{
printf ( "Smile, and the world smiles with you...\n" ) ;
}
```

- function prototype declaration: This prototype declaration indicates that **message( )** is a function which after completing its execution does not return any value.
- This ‘does not return any value’ is indicated using the keyword **void**.

The second usage of **message** is...

```
void message( )
{
    printf ( "Smile, and the world smiles with you...\n" ) ;
}
```

This is the function definition. In this definition right now we are having only **printf( )**, but we can also use **if**, **for**, **while**, **switch**, etc., within this function definition.

The third usage is...

```
message( ) ;
```

Here the function **message( )** is being called by **main( )**.

What do we mean when we say that **main( )** 'calls' the function **message( )**?

We mean that the control passes to the function **message( )**. The activity of **main( )** is temporarily suspended; it falls asleep while the **message( )** function wakes up and goes to work. When the **message( )** function runs out of statements to execute, the control returns to **main( )**, which comes to life again and begins executing its code at the exact point where it left off.

Thus, **main( )** becomes the 'calling' function, whereas **message( )** becomes the 'called' function.

```
# include <stdio.h>
void italy( ) ;
void brazil( ) ;
void argentina( ) ;
int main( )
{
printf ( "I am in main\n" ) ;
italy( ) ;
brazil( ) ;
argentina( ) ;
return 0 ;
}
        void italy( )
        {
                printf ( "I am in italy\n" ) ;
        }
        void brazil( )
        {
                printf ( "I am in brazil\n" ) ;
        }
        void argentina( )
        {
                printf ( "I am in argentina\n" ) ;
        }
```

A number of conclusions can be drawn from this program:

- A C program is a collection of one or more functions.
- If a C program contains only one function, it must be **main( )**.
- If a C program contains more than one function, then one (and only one) of these functions must be **main( )**, because program execution always begins with **main( )**.
- There is no limit on the number of functions that might be present in a C program.
- Each function in a program is called in the sequence specified by the function calls in **main( )**.
- After each function has done its thing, control returns to **main( )**.
- When **main( )** runs out of statements and function calls, the program ends.

- Write a general-purpose function to convert any given year into its Roman equivalent. Use these Roman equivalents for decimal numbers: 1 – I, 5 – V, 10 – X, 50 – L, 100 – C, 500 – D, 1000 – M.

Example:

- Roman equivalent of 1988 is mdcccclxxxviii.
- Roman equivalent of 1525 is mdxxv.

- A positive integer is entered through the keyboard. Write a function to obtain the prime factors of this number.
- For example, prime factors of 24 are 2, 2, 2 and 3, whereas prime factors of 35 are 5 and 7.